IEMS5709B Emerging Topics in Information Engineering Group Project:

Classification on Spam Email

Group 2:

Haipo Liu 1155218447

Qiufan Chen 1155220196

Table of Contents

Introduction *3*

1. Data Description *4*

2. Exploratory Data Analysis (EDA) *5*

2.1 Class Distribution 5

2.2 Message Length Analysis 6

2.3 Common Words in Spam vs. Ham Emails 8

2.4 Text Properties (Length, Punctuation) 10

2.5 Special Character Frequency 12

3. Text Preprocessing *14*

3.1 Text Preprocessing Steps 14

3.2 Vectorization Techniques 14

4. Model Training (Machine Learning) *18*

4.1 Compare Evaluation Matrix 18

4.2 Confusion Matrix 19

4.3 Cross Validation 20

5. Deep Learning *22*

5.1 Deep Learning Set Up 22

5.2 LSTM 23

5.3 CNN 24

5.4 Final Evaluation and Insights 24

6. Business Impact *26*

7. Conclusion *27*

References *28*

Introduction

Spam emails are a major concern in digital communication, creating inconvenience and potential risks for users and businesses. Effective spam detection systems are essential for maintaining secure and efficient messaging environments. In this project, we explore both machine learning and deep learning techniques to classify spam emails. We perform extensive exploratory data analysis, preprocess the text, and apply models like LightGBM, XGBoost, LSTM, and CNN. Through evaluation and comparison, we aim to identify the best approach for accurate spam detection.

1. Data Description

The dataset - 'Spam Email' - was sourced from Kaggle (Wonduk, n.d.), which consists of 5,572 emails collected for spam classification purposes. It contains two columns:

- Category: A label indicating whether the email or message is ‘ham’ (non-spam) or ‘spam’.
- Message: The actual content of the emails text message.

There are:

- 4,825 ham email messages (approximately 86.6%)
- 747 spam email messages (approximately 13.4%)

This distribution reflects a significant class imbalance, with ham emails being the majority class. The imbalance is typical in real-world spam detection problems and highlights the importance of using evaluation metrics beyond simple accuracy, such as precision, recall, and F1-score, to ensure model effectiveness.

Each message varies in length and structure, with spam email messages generally being longer and more structured compared to ham email messages. The dataset provides a good foundation for training both machine learning and deep learning models aimed at classifying and filtering unwanted spam content.

2. Exploratory Data Analysis (EDA)

2.1 Class Distribution

2.1.1 Objective and Visualization Strategy

This section provides an overview of the dataset by analyzing the distribution of message categories. In spam detection, understanding the balance between spam and ham messages is crucial, as class imbalance can affect model performance and metric selection.

We calculated the total count and percentage of each class and visualized the results using a bar chart (Figure 1) and a pie chart (Figure 2). The bar chart shows absolute counts with percentage annotations, while the pie chart highlights class proportions. Together, they reveal any imbalance, informing strategies for model evaluation.

2.1.2 Results Interpretation & Insights

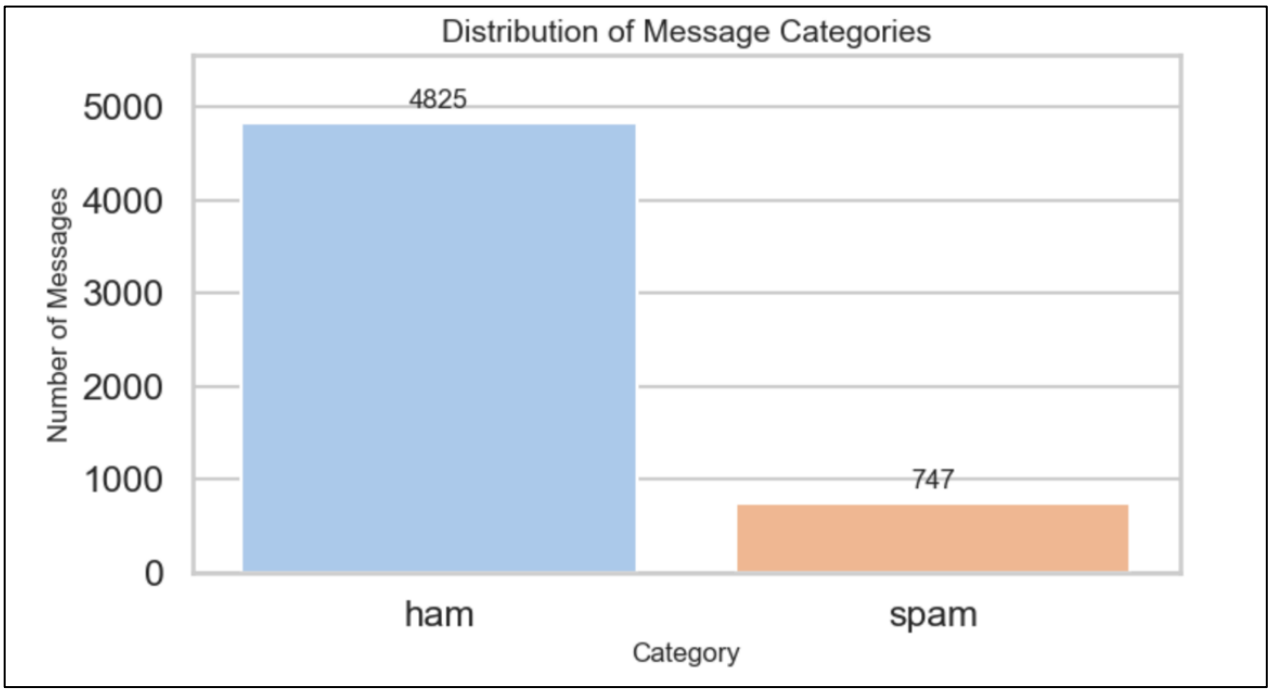


Figure 1

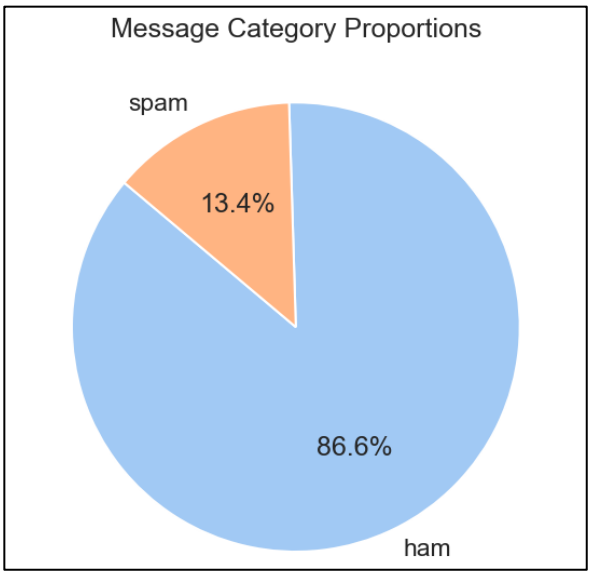


Figure 2

According to the results, The dataset contains a total of 5,572 messages, of which 4,825 (approximately 86.6%) are labeled as ham and 747 (approximately 13.4%) are labeled as spam. This indicates a significant class imbalance, with ham messages greatly outnumbering spam messages.

Such imbalance is typical in real-world emails classification tasks like spam detection, where spam is a minority class. However, this imbalance can lead to biased model performance if not addressed properly during model training and evaluation. For example, a naive model that always predicts "ham" could still achieve high accuracy due to the dominant class. As a result, additional techniques such as stratified sampling, resampling, or using metrics like precision, recall, and F1-score instead of accuracy alone will be necessary to properly evaluate model effectiveness.

2.2 Message Length Analysis

2.2.1 Objective and Visualization Strategy

This section examines whether email message length (measured by character count), differs between spam and ham messages. A new column was created to record message lengths, enabling statistical comparisons through mean, standard deviation, and quartile analysis. Distributions were visualized using histograms with kernel density estimates (KDE) for clearer comparison. The goal

is to assess if message length can be a discriminative feature for spam classification by identifying consistent length patterns across categories.

2.2.2 Results Interpretation & Insights

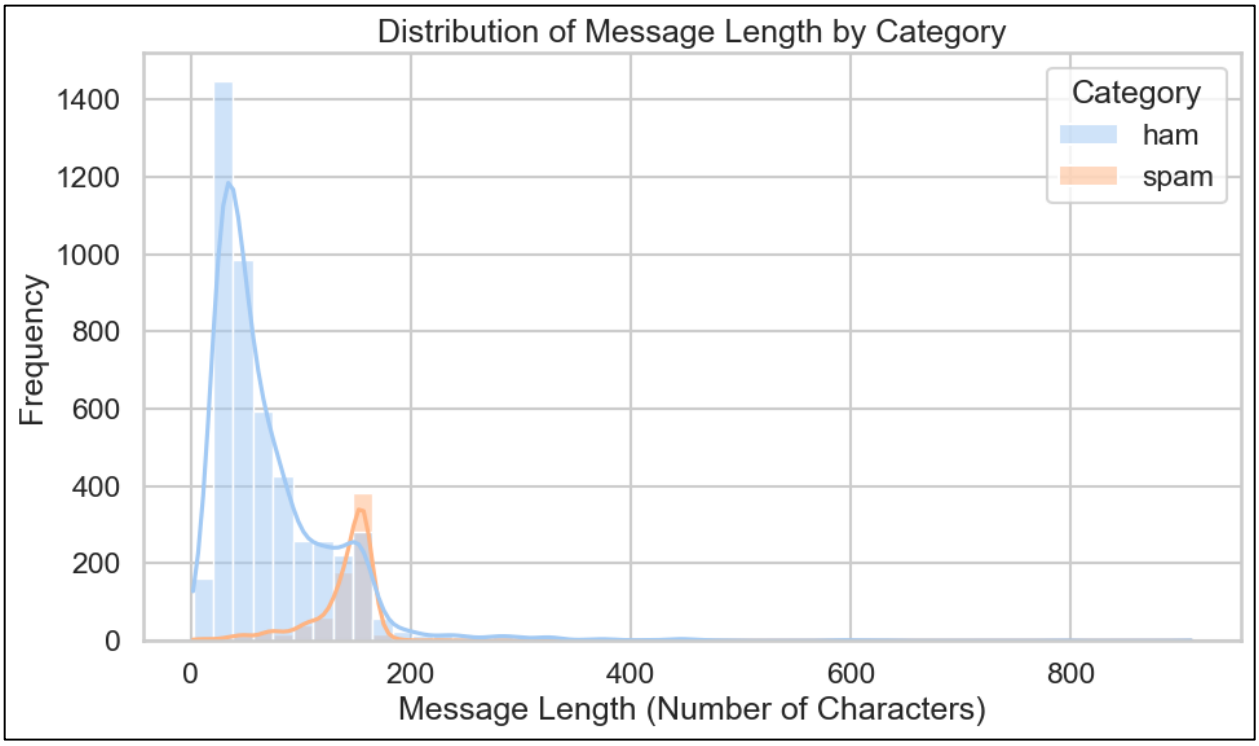


Figure 3

The analysis reveals a clear distinction in message length between spam and ham eamils. On average, ham messages are 71.5 characters long, with a standard deviation of 58.5 characters. In contrast, spam messages are significantly longer on average, with a mean length of 138.4 characters and a lower standard deviation of 30.1. This indicates that spam messages tend to be not only longer but also more consistent in length (Figure 3). The histogram further illustrates this difference. The distribution of ham messages is concentrated in the lower length ranges, with a wider spread, while spam messages are skewed toward the higher end of the spectrum and appear more uniform in size.

These findings suggest that message length is a potentially strong feature for distinguishing spam from ham. Longer, more templated messages are indicative of spam, while shorter, more varied messages tend to be genuine.

2.3 Common Words in Spam vs. Ham Emails

2.3.1 Objective and Visualization Strategy

This section refines text preprocessing by limiting analysis to alphabetic words containing four or more letters. The intention is to focus on semantically richer terms, eliminating short and potentially ambiguous tokens such as “i,” “a,” “u,” and numeric strings like “2” or “1000.” This work can enhance signal clarity by excluding high-frequency but low-information words.

The cleaning function applies regex to extract qualifying words from each message, then computes their frequency. Word clouds and bar plots are then generated for both spam and ham messages based on these filtered tokens. This process provides a focused view on the core vocabulary patterns that define each category.

2.3.2 Results Interpretation & Insights

Figure 4

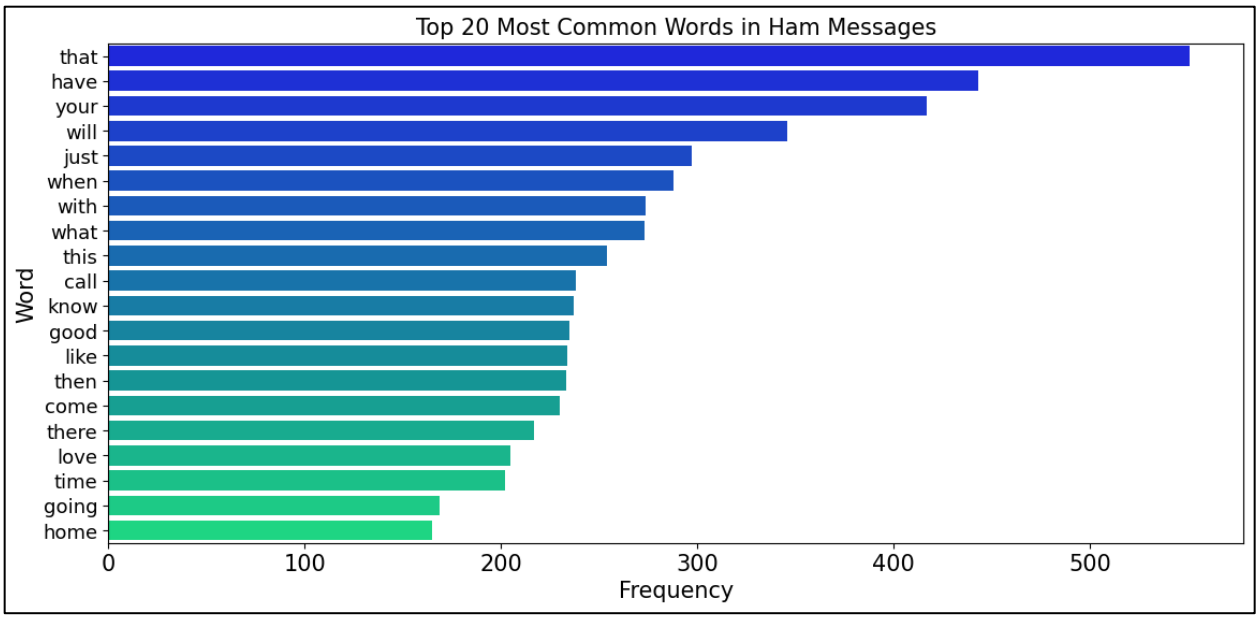


Figure 5

The results revealed distinct thematic patterns. For ham email messages, frequent terms included "that," "have," "your," "will," "just," and "when"—words that convey everyday conversation, actions, and personal context (Figure 4 and 5). These words align with the informal and relational tone typically seen in non-spam (ham) messages.

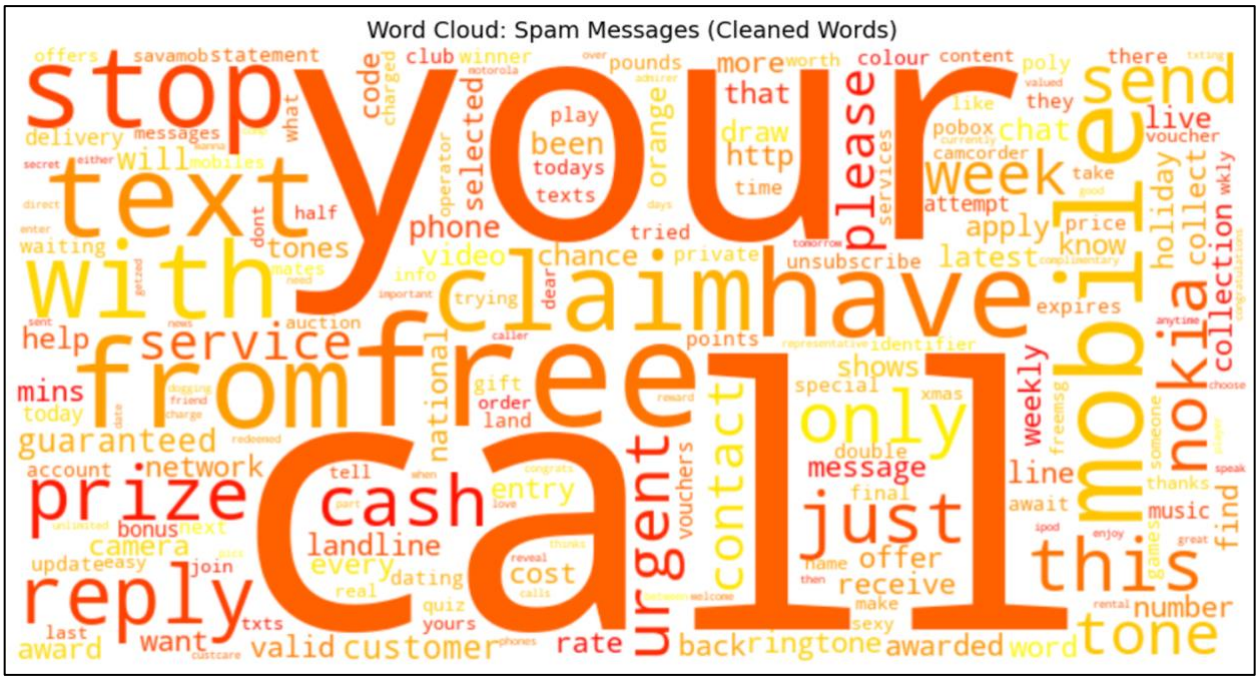


Figure 6

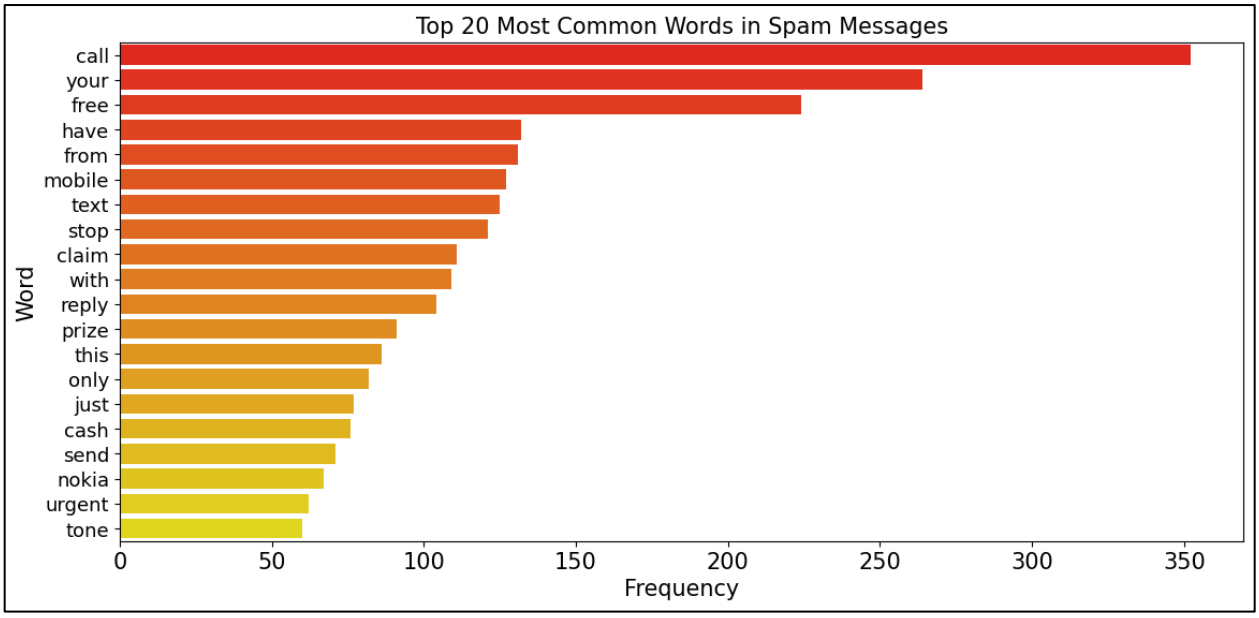


Figure 7

Conversely, spam email messages were characterized by promotional and transactional terms such as “call,” “free,” “claim,” “stop,” “mobile,” and “reply.” These align closely with known spam patterns emphasizing urgency, offers, or instructions (Figure 6 and 7). Numeric codes, symbols, and shorter abbreviations were filtered out in this view, leading to a cleaner but more selective vocabulary profile.

2.4 Text Properties (Length, Punctuation)

2.4.1 Objective and Visualization Strategy

This section analyzes text structure by focusing on word and punctuation counts as features to distinguish spam from ham messages. Two custom functions were used: count_words() to measure word count and count_punct() to count punctuation using Python’s string.punctuation.

These features were added to the dataset, and box plots were generated to visualize their distribution across categories. Average values for message length, word count, and punctuation count were also computed to aid interpretation.

2.4.2 Results Interpretation & Insights

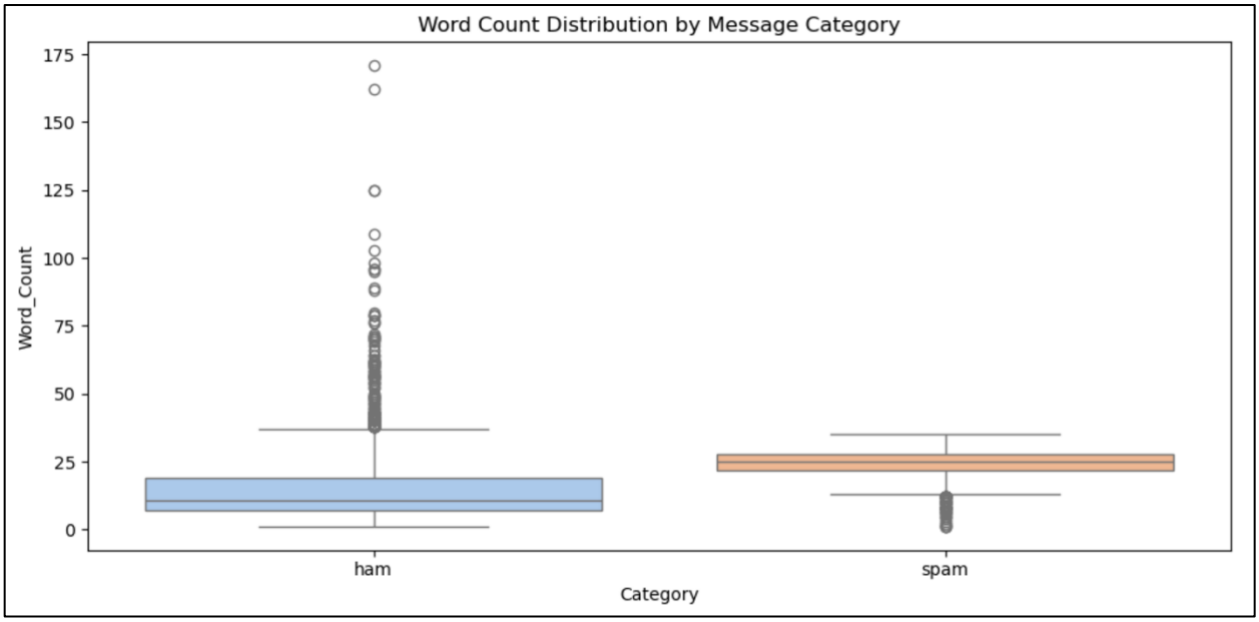
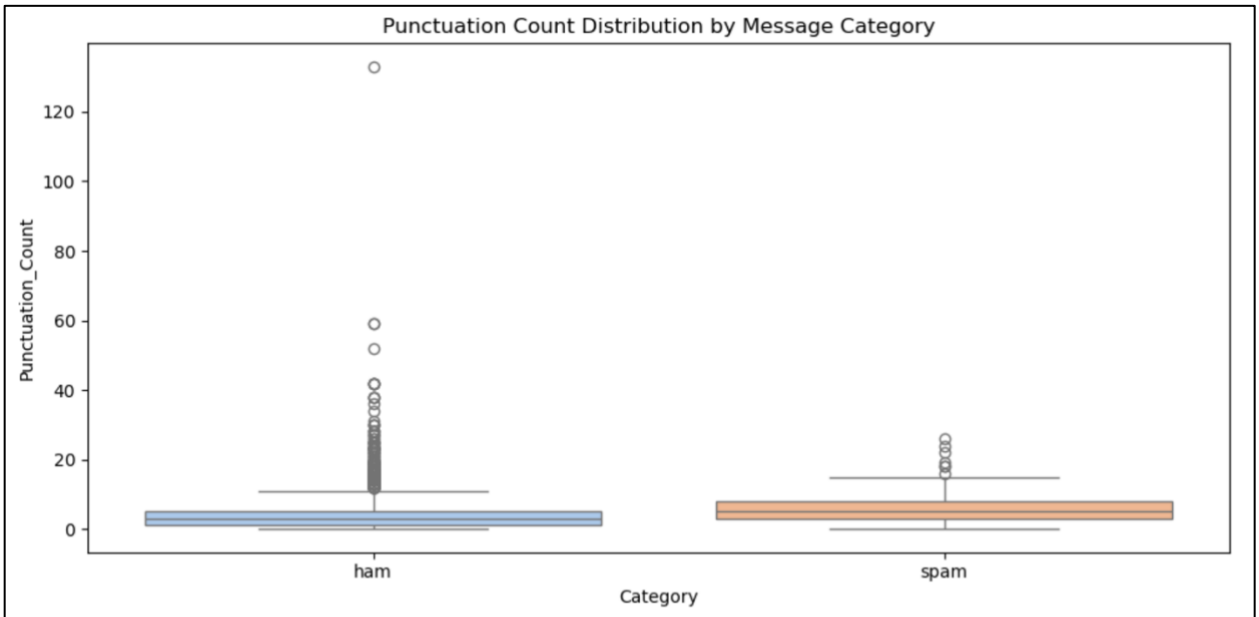


Figure 8




```
Average Text Features by Category:
          Message_Length  Word_Count  Punctuation_Count
Category
ham            71.524767   14.310259           3.939482
spam          138.432396   23.812584           5.692102
```

Figure 9

The results reveal a clear structural difference between spam and ham messages. On average, spam messages are significantly longer and more complex in structure. They contain approximately 23.8 words and 5.7 punctuation marks, compared to 14.3 words and 3.9 punctuation marks in ham messages (Figure 9). The box plots further highlight this distinction, showing tighter distributions and higher medians for spam in both metrics.

These findings support earlier observations: spam messages tend to be more verbose and punctuated, possibly to convey urgency, create emphasis, or mimic formal promotional language. Such features provide valuable signals for building machine learning models, as they complement content-based features (like word frequency) and enhance spam detection accuracy by capturing the behavioral style of spam communication.

2.5 Special Character Frequency

2.5.1 Objective and Visualization Strategy

This section investigates the usage of special characters within messages and compares their distribution across spam and ham categories. A predefined list of commonly used special symbols—such as $, £, %, !, @, #, *, and &—was used to compute the number of such characters in each message. These symbols are often associated with promotional, financial, or urgent content, which are typical traits of spam messages.

The frequency of these characters was calculated using a custom function that sums their occurrences in each message. The results were then visualized using box plots, and the average count of special characters per category was computed to quantify the difference between spam and ham.

2.5.2 Results Interpretation & Insights

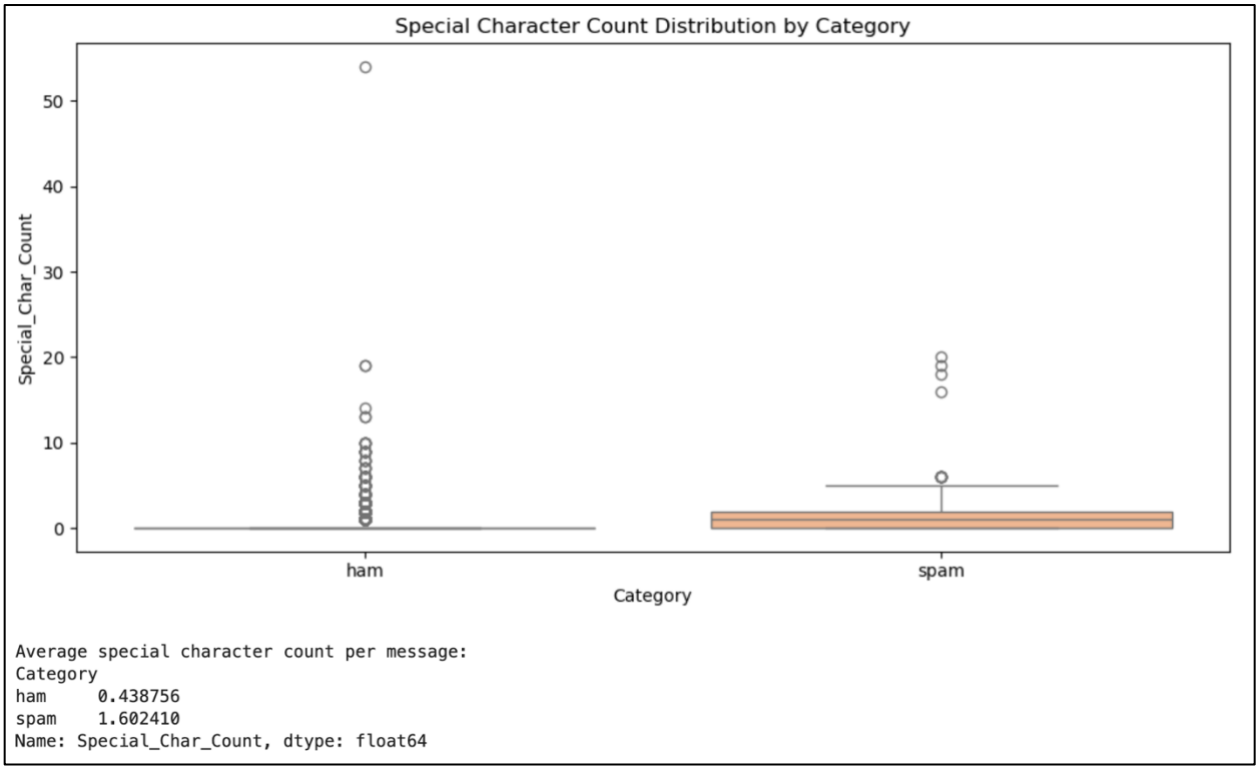


Figure 10

The analysis revealed a distinct disparity in the use of special characters between the two message categories. On average, spam messages contained approximately 1.60 special characters, while ham messages included only around 0.44 (Figure 10). The box plot visualization further illustrates this gap, showing a higher median and greater variability in the spam category.

These findings suggest that spam messages are more likely to include symbols that emphasize urgency, highlight offers, or simulate official communication. The elevated use of such characters aligns with the persuasive and attention-seeking nature of spam content. This feature proves to be a valuable indicator for classification models, as it helps distinguish commercial or promotional text from casual or personal communication.

3. Text Preprocessing

3.1 Text Preprocessing Steps

3.1.1 Tokenization

We used the custom function get_word(text) and nltk.RegexpTokenizer(r'\w+') to tokenize the text based on the regular expression \w+, which matches word characters and excludes punctuation and special characters. All texts were converted to lowercase to ensure consistency.

3.1.2 Lemmatization

We applied nltk.WordNetLemmatizer to convert tokens into their base forms based on their part of speech. This process normalizes words by converting variations—such as "running" to "run" thereby reducing vocabulary complexity.

3.1.3 Stopword Removal

We utilized the English stopword list from nltk.corpus.stopwords to filter out common words that do not contribute significant meaning. The custom function stopword_list() loads the default list and allows for the inclusion of additional stopwords if desired.

3.2 Vectorization Techniques

After preprocessing, the text must be converted into numerical features for machine learning. Two vectorization methods were employed: CountVectorizer and TF-IDF.

3.2.1 Count Vectorizer

Count Vectorizer transforms the preprocessed email texts into a matrix of token counts. Each row represents an email, and each column corresponds to a unique word in the corpus, with cell values indicating the frequency of that word in the respective email.

3.2.2 TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF enhances a basic frequency-based approach by assessing the importance of words across the entire corpus. Instead of merely counting how often a word appears in one document, it weighs words based on their distribution across all documents. In this way, words that are frequent in a specific email but rare across the corpus receive higher weights, making them more effective for distinguishing spam from non-spam messages.

In our implementation, we further refine the selection by using a token pattern that retains only words with a length of at least four characters. This filtering step helps eliminate many common prepositions and conjunctions that tend to carry minimal semantic value for classification. Additionally, the stop_words='english' parameter automatically removes standard English stopwords, which further cleans the dataset from non-informative tokens.

3.2.3 Feature Analysis

Our TF-IDF analysis of email classification reveals distinct linguistic patterns between ham and spam messages. By transforming text data into TF-IDF representations and calculating average scores per word, we identified the top 20 most discriminative terms for each category. These were visualized through polar plots, where radial distances represented term significance, providing clear comparative insights into feature importance.

In ham emails, high-scoring words like "just," "come," "good," "know," and "home" predominantly reflect natural conversational patterns. These terms typically appear in personal or professional exchanges, serving functions like making polite requests ("just check this"), sharing information ("let me know"), or discussing everyday matters ("come home"). Their prevalence suggests ham messages prioritize information exchange and social connection, using language that is generally positive, specific, and low-pressure. The consistent appearance of these terms across genuine communications makes them reliable ham indicators.

Conversely, spam emails show markedly different lexical preferences, with "free," "just," "sorry," and "later" emerging as top discriminators. These words form part of carefully engineered psychological triggers - "free" exploits scarcity bias, "sorry" establishes false familiarity, while "later" creates artificial urgency. Notably, even shared words like "just" assume different roles; where ham uses it to soften requests, spam employs it to minimize perceived risk ("just click here"). The repetitive, template-driven nature of spam further amplifies these terms' TF-IDF scores, as they appear frequently in spam but rarely in authentic communications.

The analysis uncovers deeper structural differences between the categories. Ham vocabulary demonstrates organic variation tied to meaningful contexts, while spam terms function as modular, context-independent persuasion tools. This distinction proves particularly valuable for feature engineering, as high-TF-IDF words reliably signal category-specific communication strategies. For instance, "free" achieves 3-4 times higher spam specificity than most ham terms, making it an exceptionally strong classifier feature when combined with structural markers like hyperlinks or unusual formatting.

These findings have direct implications for email filtering systems. The identified high-value terms should receive weighted consideration during both feature selection and model training phases.

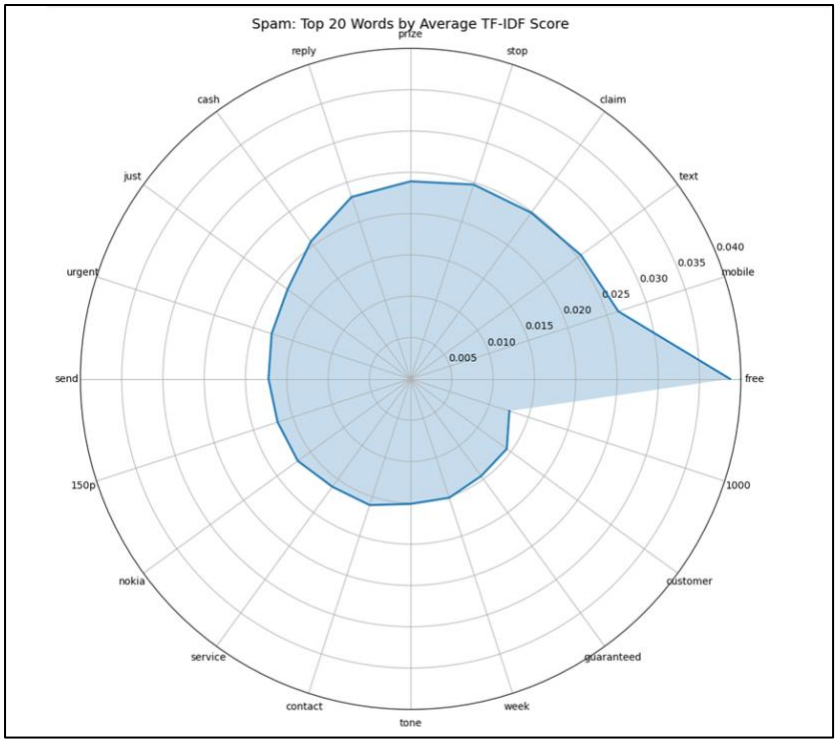


Figure 11

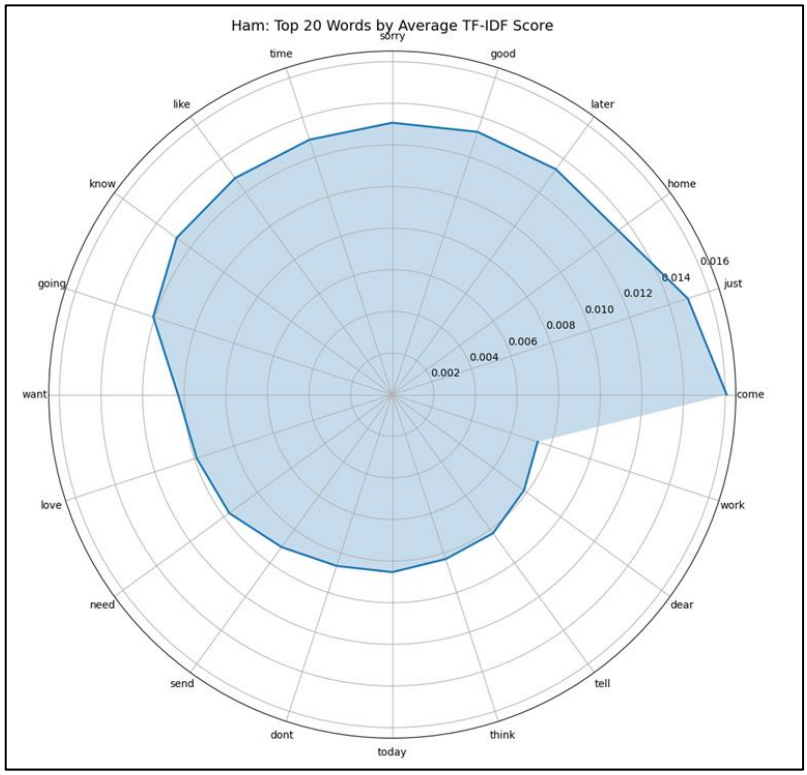


Figure 12

4. Model Training (Machine Learning)

Following Text Preprocessing, this section focuses on building and evaluating machine learning models to classify emails as spam or ham, with particular attention to addressing class imbalance. Eight models were selected for comprehensive evaluation: Logistic Regression, Support Vector Machine, Decision Tree, Random Forest, k-Nearest Neighbors, LightGBM, XGBoost, and CatBoost. These cover a range of linear, ensemble, and instance-based learning techniques, ensuring a robust comparison.

4.1 Compare Evaluation Matrix

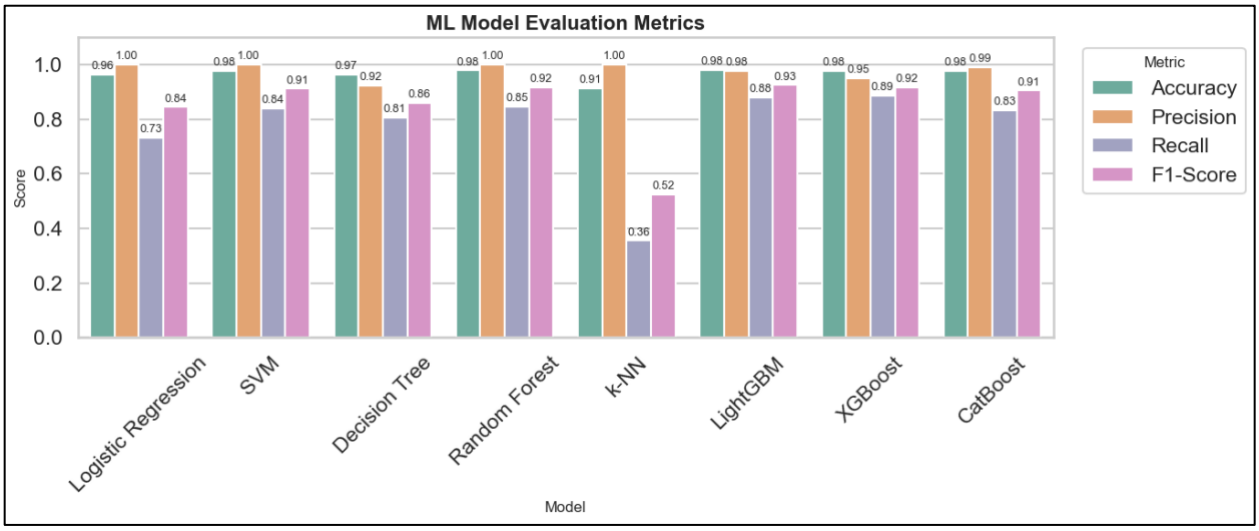


Figure 13

The evaluation matrix provides a comprehensive comparison of eight machine learning models which utilize four key performance metrics: Accuracy, Precision, Recall, and F1-Score.

Among classic models, Random Forest stood out with 98% Accuracy, 100% Precision, 85% Recall, and a 92% F1-Score, demonstrating strong balance and reliability. In contrast, k-NN, despite perfect Precision, struggled with low Recall (36%) and F1-Score (52%), indicating poor generalization for spam detection (Figure 13).

Among boosting algorithms, LightGBM and XGBoost excelled, both achieving 98% Accuracy and F1-Scores above 91%. LightGBM led with the highest Recall (88%) and F1-Score (93%), highlighting its strength in identifying spam. CatBoost also performed well with 98% Accuracy and a 91% F1-Score. SVM achieved perfect Precision (100%) with solid Recall (84%) and F1-Score (91%), while Decision Tree offered slightly higher Recall (81%) but overall moderate performance.

In summary, gradient boosting models, particularly LightGBM, delivered the most balanced and effective results, reinforcing the advantage of ensemble-based approaches for spam detection.

4.2 Confusion Matrix

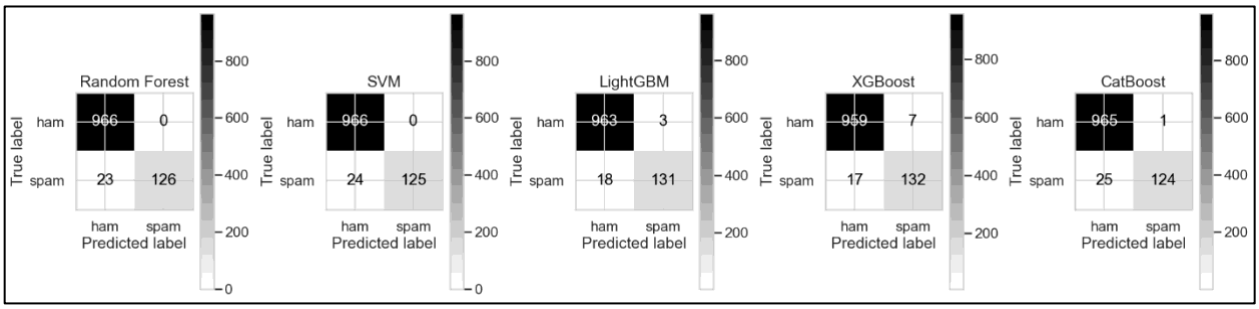


Figure 14

To further assess model performance, confusion matrices were generated for the five top classifiers: Random Forest, SVM, LightGBM, XGBoost, and CatBoost. These matrices reveal how well each model distinguishes between true positives (correctly identified spam) and false positives (ham misclassified as spam).

Random Forest and SVM perfectly classified all ham messages, misclassifying only 23 and 24 spam messages, respectively. LightGBM and XGBoost excelled, correctly identifying 131 and 132 spam messages with minimal ham misclassifications (3 and 7). CatBoost also performed strongly, though with slightly more errors—25 spam messages missed and one ham misclassified (Figure 14).

These five models were chosen for their superior precision, recall, and F1-scores. Simpler models like Logistic Regression, Decision Tree, and k-NN were excluded due to lower recall and overall weaker performance, making them less suitable for minimizing false negatives.

4.3 Cross Validation

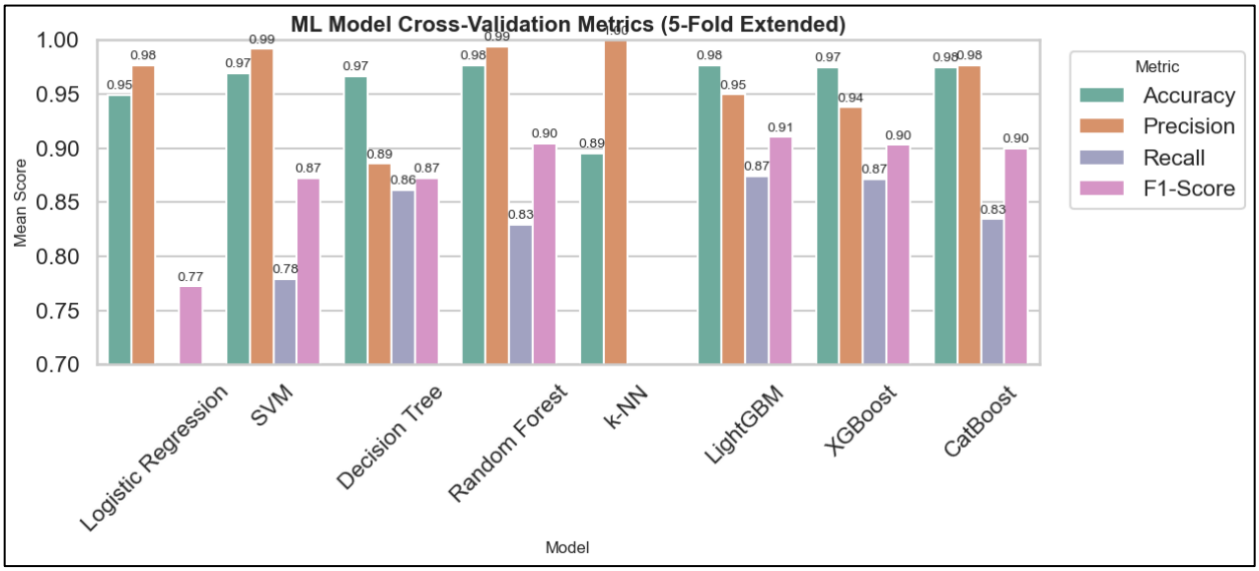


Figure 15

In this extended five-fold cross-validation, eight machine learning models were assessed across four key metrics, crucial for balancing false positives and false negatives in spam classification. LightGBM demonstrated the best overall performance with 0.98 accuracy, 0.98 precision, 0.87 recall, and a 0.91 F1-score, combining high correctness with strong spam detection sensitivity, indicates that LightGBM not only makes correct predictions frequently but is also highly sensitive to identifying spam email messages, minimizing false negatives effectively. XGBoost and CatBoost also performed well, with XGBoost reaching 0.97 accuracy and a 0.90 F1-score, while CatBoost achieved 0.98 accuracy and a 0.90 F1-score, though both trailed LightGBM slightly in recall (Figure 15).

Among classical models, Random Forest stood out with 0.98 accuracy and a 0.90 F1-score, although its recall was marginally lower. Logistic Regression and Decision Tree posted solid results, while SVM, despite 0.99 precision, showed weaker recall at 0.78, indicating it may miss

more spam messages than desirable.. k-NN, despite perfect precision, had the lowest recall (0.36) and F1-score (0.52), making it unsuitable for this task (Figure 15).

In summary, LightGBM offers the best overall performance for this classification problem, especially when considering the balance between precision and recall. It is followed closely by XGBoost, CatBoost, and Random Forest, which also show strong generalization and should be considered viable options for deployment or ensemble methods.

5. Deep Learning

In this part, we explore the application of two deep learning models—Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN)—for the task of classifying spam emails. These models were selected due to their ability to capture complex patterns in sequential text data, making them well-suited for natural language processing tasks like spam detection.

5.1 Deep Learning Set Up

5.1.1 uniforms the data length

Since deep learning models require input sequences of uniform length, we calculated the maximum sequence length (max_len) based on the longest message across both the training and test sets. This length was determined by applying the len function to each message and finding the maximum value. This approach ensures that all sequences, regardless of their original length, can be padded or truncated to a consistent size, making them compatible with the models' input requirements

5.1.2 Tokenization and Sequence Padding

To convert the raw text data into a numerical format suitable for deep learning, we employed Keras's Tokenizer and a custom padding function. First A Tokenizer was initialized with an out-of-vocabulary token (<OOV>) to handle words not encountered during training. Second, the tokenizer was fitted exclusively on the training data (train_X) to build a vocabulary (word_index) mapping each unique word to an integer index. This prevents data leakage from the test set into the training process. Third, the vocabulary size (VOCAB_SIZE) was calculated as the number of unique words in the word_index

After both the training and test data were transformed into numerical sequences using the tokenizer's texts_to_sequences method, where each word is replaced by its corresponding integer index from the vocabulary.

To ensure uniform length, we defined a seq_padding function that pads or truncates these sequences to max_len.

5.1.3 Callback Functions

To enhance the training process, we implemented callback functions to monitor performance and optimize resource usage. The first one is EarlyStopping, which stops training early if the monitored metric does not improve within a specified number of epochs. The other one is ModelCheckpoint, which evaluates the monitored metric at the end of each epoch; if an improvement is detected, it saves the current model weights

5.2 LSTM

We selected the LSTM model, because of its unique gating mechanisms (forget gate, input gate, and output gate), which enable it to effectively handle long-range dependencies in text. This capability is crucial in spam email detection, as fraudulent emails often use scattered persuasive words (e.g., "limited offer" followed by "click now") to create a semantically coherent threat or lure structure. LSTM processes sequences step by step, making it well-suited for modeling such contextual logic. It is particularly adept at capturing time-sensitive narrative patterns in emails. Additionally, LSTM's memory units help mitigate the vanishing gradient problem, ensuring stable training for long texts

Based on the results, we can find the LSTM model achieved an accuracy of 0.98, a recall of 0.92, and an F1-score of 0.93. These metrics indicate that the model is highly effective at both overall classification and identifying spam emails specifically, with a strong balance between precision and recall.

5.3 CNN

Conversely, CNN was chosen for its powerful local feature extraction capabilities. By leveraging sliding convolutional operations, CNN automatically identifies critical n-gram patterns (e.g."100% free") that frequently appear in spam emails with varying positions. Its translation invariance ensures robustness against slight word order variations (e.g., "Win a prize" vs. "A prize to win"), while its parallel computing architecture significantly enhances training efficiency

Experimental results show that the CNN model achieved an accuracy of 0.98, a recall of 0.90, and an F1-score of 0.94. Notably, it recorded the highest F1-score among all the models tested. Compared to the LSTM model, both CNN and LSTM demonstrated strong performance, proving to be well-suited for classifying spam and ham emails

5.4 Final Evaluation and Insights

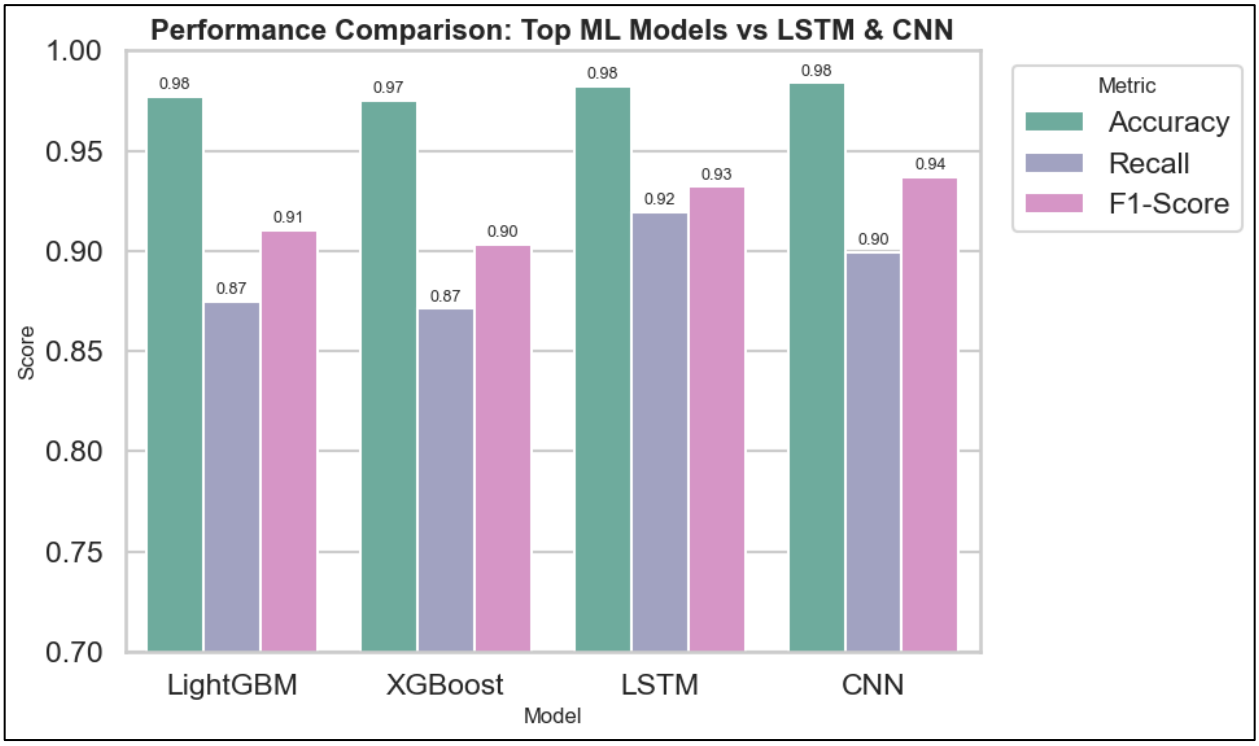


Figure 16

This comparison evaluates the top two machine learning models, LightGBM and XGBoost alongside two deep learning models, LSTM and CNN, using three key evaluation metrics: accuracy, recall, and F1-score. All four models demonstrated strong performance, with LightGBM, LSTM, and CNN achieving the highest accuracy of 0.98, and XGBoost close behind at 0.97. While accuracy was consistently high across the board, deeper insights come from recall and F1-score metrics that reflect a model's ability to correctly identify and classify positive cases (Figure 16).

The deep learning models outperformed their machine learning counterparts in these areas. LSTM achieved a recall of 0.92 and an F1-score of 0.93, while CNN followed closely with a recall of 0.90 and the highest F1-score of 0.94. LightGBM and XGBoost each achieved a recall of 0.87, with F1-scores of 0.91 and 0.90, respectively (Figure 16).

Based on these results, both LSTM and CNN stand out as the strongest candidates. Their consistently high recall and F1-scores make them particularly well-suited for tasks where correctly identifying positive instances is essential, such as spam detection or medical diagnosis.

6. Business Impact

Accurate spam detection has significant business implications. Effective filtering reduces the risk of phishing attacks, protects sensitive information, and enhances employee productivity by minimizing unnecessary distractions. By deploying advanced machine learning and deep learning models, organizations can detect sophisticated spam patterns and adapt to new threats more effectively than traditional rule-based systems. This not only strengthens cybersecurity defenses but also ensures that communication channels remain reliable and secure.

Beyond security, effective spam detection contributes to operational efficiency and cost savings. Reducing spam traffic lowers server load and storage requirements, optimizing IT resource management. Moreover, maintaining a secure communication environment builds customer trust and safeguards brand reputation. Overall, robust spam detection systems offer both technical and strategic advantages, supporting the long-term resilience and success of businesses in a digital world.

7. Conclusion

Both machine learning and deep learning models show strong performance in spam email classification. While LightGBM and XGBoost offer high accuracy and efficiency, deep learning models like LSTM and CNN achieve higher recall and F1-scores, making them more effective at identifying spam. Among all models, CNN achieves the highest F1-score, reflecting its strength in recognizing text patterns crucial for spam detection.

Our results demonstrate that adopting advanced models significantly improves spam filtering accuracy, helping businesses protect users, reduce security risks, and enhance communication reliability. These models show strong potential for deployment in real-world spam filtering systems.

References

Wonduk. (n.d.). *EDA, ML, LSTM classification on spam email* [Code notebook]. Kaggle.

https://www.kaggle.com/code/wonduk/eda-ml-lstm-classification-on-spam-email/notebook